FDS-Vertretungsplan

Entwicklung einer webbasierten Vertretungsplan-Anwendung
für die Friedrich-Dessauer-Schule Limburg

Schriftliche Ausarbeitung zur Besonderen Lernleistung im Landesabitur 2026

Name	Lukas Dienst
Schule	Friedrich-Dessauer-Schule Limburg
Referenzfach	Informatik
Betreuende Lehrkraft	Alexander Rhode
Zweitprüfer	Michael Knobl
Schuljahr	2025 / 2026
Abgabedatum	[eintragen]

Kurzfassung

Ausgangspunkt dieser besonderen Lernleistung war die Beobachtung, dass der offizielle Vertretungsplan der Friedrich-Dessauer-Schule Limburg zwar digital über WebUntis verfügbar ist, im schulischen Alltag jedoch nur eingeschränkt nutzbar war. Die Plattform setzt einen schulischen Account voraus, ist auf mobilen Geräten wenig komfortabel und bietet keine alltagstaugliche Filterung nach individuellen Fächern oder Kursen.

Ziel des Projekts war daher nicht die Entwicklung einer neuen Datenquelle, sondern die Konzeption und Umsetzung einer eigenständigen Webanwendung, die bestehende Vertretungsdaten verständlicher, schneller und personalisiert zugänglich macht. Im Ergebnis entstand eine Anwendung, die Vertretungen für ein gewähltes Datum übersichtlich darstellt, leistungsfähige Such- und Filterfunktionen bietet, persönliche Stundenpläne berücksichtigt und relevante Änderungen per Push-Benachrichtigung meldet.

Die Entwicklung erfolgte iterativ in drei Phasen: Eine erste statische Version ohne Backend verdeutlichte die Grenzen eines filterfreien Ansatzes. Eine zweite Version auf Basis von Next.js zeigte die Notwendigkeit einer klaren Trennung von Frontend und API sowie einer strukturierten Komponentenarchitektur. Die dritte und finale Version wurde vollständig neu konzipiert. Sie integriert eine Nutzerauthentifizierung, eine relationale Datenbank, einen cachegestützten API-Proxy sowie ein delta-basiertes Push-Benachrichtigungssystem.

Technisch wurde die Anwendung mit Next.js (App Router), React, TypeScript, NextAuth, Prisma und PostgreSQL umgesetzt. Eine besondere Herausforderung stellte die Erschließung der WebUntis-Schnittstelle durch systematische Netzwerkanalyse dar. Darauf aufbauend wurden ein stabiler API-Proxy mit Caching- und Fehlerstrategien sowie eine eigene Delta-Logik zur Erkennung relevanter Änderungen entwickelt. Insgesamt sichern 134 automatisierte Testfälle die zentrale Systemlogik ab.

Die Arbeit dokumentiert nicht nur das entstandene Produkt, sondern insbesondere den gesamten Entwicklungsprozess - von der Anforderungsanalyse über Architekturentscheidungen bis hin zu aufgetretenen Problemen, deren Lösungen sowie der abschließenden Bewertung der gesetzten Ziele. Die fünf definierten SMART-Ziele wurden dabei größtenteils erreicht: Die vier Kernziele (S1-S4) vollständig, das Ziel der Wartbarkeit weitgehend.

Die besondere Lernleistung im Fach Informatik umfasst die Konzeption, Implementierung und reflektierte Dokumentation einer vollständigen Webanwendung.

Kurzfassung 2
1 Einleitung und Motivation 5
1.1 Ausgangssituation 5
1.2 Motivation und Themenwahl 5
1.3 Warum eine Webanwendung? 6
1.4 Einordnung als Besondere Lernleistung 7
1.5 Formale Eckpunkte der BLL 7
1.6 Aufbau der Arbeit 8
2 Informationsquellen und Anforderungen (Lastenheft) 8
2.1 Woher stammen die Anforderungen? 8
2.2 Zielgruppen und Beteiligte 8
2.3 Funktionale Anforderungen 9
2.4 Nicht-funktionale Anforderungen 9
2.5 Bedienung im Schulalltag 9
2.6 SMART-Ziele 10
2.7 Zentrale Anwendungsfälle 11
3 Lösungsansatz und Pflichtenheft 11
3.1 Grundidee der Lösung 11
3.2 Architekturprinzip 11
3.3 Öffentliche Nutzung, Registrierung und Dashboard 12
3.4 Rollen und Berechtigungen 12
3.5 Datenbeschaffung über WebUntis 12
3.6 API-Proxy, Caching und Fehlerpfade 13
3.7 Push-Benachrichtigungen und Delta-Logik 13
3.8 Warum 15-Minuten-Polling statt Echtzeit? 13
3.9 Hosting und Bereitstellung 14
4 Projektmanagement und Zeitplanung 14
4.1 Iteratives Vorgehen statt starrem Vorgehensmodell 14
4.2 Entwicklungsversionen 14
4.3 Meilensteine4.4 Gantt-Plan 15
4.5 Netzplan und kritischer Pfad 16
4.6 Rückmeldungen und Anpassungen 17
4.7 Methodische Reflexion 17
5 Tools und Tech Stack 18
5.1 Auswahlkriterien 18
5.2 Kerntechnologien 18
5.3 Warum Framework statt reinem HTML / CSS? 19
5.4 Passung des Stacks zum Projekt 19
6 Systemstruktur, UML und Abläufe 20
6.1 Use-Case-Diagramm 20
6.2 Modulorientiertes Klassendiagramm 21
6.3 Objektdiagramm 22
6.4 Aktivitätsdiagramm: Abruf der Vertretungen 23

6.5 Sequenzdiagramm: Nutzerabruf 24
6.6 Sequenzdiagramm: Push-Dispatch 25
6.7 Bedeutung der Diagramme für die Arbeit 25
7 Technische Umsetzung und Schwierigkeiten 26
7.1 Aufbau des Repositories 26
7.2 Zentrale Module und ihre Aufgaben 26
7.3 Datenverarbeitung und Filterlogik 27
7.4 Nutzerfluss und Bedienbarkeit 27
7.5 Sicherheit, Datenschutz und Betrieb 27
7.6 Typische Schwierigkeiten und dessen Lösungen 28
7.7 Lessons Learned 28
8 Test und Ergebnisbewertung 29
8.1 Teststrategie 29
8.2 Ausgewählte Testfälle 29
8.3 Soll-Ist-Abgleich der SMART-Ziele 30
8.4 Grenzen der Lösung 31
9 Fazit und Ausblick 31
9.1 Zusammenfassung 31
9.2 Weiterentwicklung 31
9.3 Persönlicher Lerngewinn 32
10 Quellenverzeichnis 33
10.1 Projektinterne Quellen 33
10.2 Technische Dokumentationen 33
10.3 Webstandards und Sicherheitsquellen 33
10.4 Fachliteratur 33
10.5 Organisatorische Quellen 34

1 Einleitung und Motivation

1.1 Ausgangssituation

Vertretungspläne gehören zu den zentralen Informationsquellen des Schulalltags. Gerade bei kurzfristigen Änderungen ist es wichtig, dass betroffene Informationen schnell, eindeutig und ohne unnötige Zwischenschritte abrufbar sind. In vielen Schulen liegen diese Daten zwar bereits digital vor, ihre tatsächliche Nutzbarkeit bleibt jedoch oft hinter den Anforderungen des Alltags zurück. Typische Probleme sind lange, schwer überschaubare Tabellen, fehlende mobile Optimierung sowie unzureichende Möglichkeiten, gezielt nach bestimmten Klassen, Kursen oder Fächern zu filtern.

Auch an der Friedrich-Dessauer-Schule Limburg war der Vertretungsplan digital über WebUntis verfügbar. Die Darstellung erfolgte dort jedoch als lange, automatisch durchlaufende Ansicht ohne Suchfunktion und ohne Möglichkeit, gezielt nach der eigenen Klasse, einem bestimmten Kurs oder einem konkreten Fach zu filtern. Wer lediglich wissen wollte, ob ein eigener Eintrag betroffen war, musste die gesamte Anzeige beobachten und die relevanten Zeilen selbst herausfiltern. Insbesondere auf dem Smartphone erwies sich diese Form der Darstellung als unübersichtlich und nur eingeschränkt alltagstauglich.

Genau an dieser Stelle setzte das Projekt an: Nicht die Vertretungsdaten selbst sollten verändert, sondern deren Zugänglichkeit, Übersichtlichkeit und Personalisierbarkeit deutlich verbessert werden.

1.2 Motivation und Themenwahl

Die Motivation für dieses Projekt entstand direkt aus dem Schulalltag. Ziel war es bewusst, kein rein theoretisches Informatikprojekt zu entwickeln, sondern ein praxisnahes System, das ein konkretes Problem an der eigenen Schule löst. Der Vertretungsplan bot sich hierfür besonders an, da die zugrunde liegenden Daten bereits vorhanden waren, deren Nutzwert im Alltag jedoch deutlich ausbaufähig war.

Das Projekt fds-vertretungsplan setzt genau an dieser Stelle an. Ziel ist nicht die Erschließung einer neuen Datenquelle, sondern die Entwicklung einer Anwendung, die die vorhandenen Vertretungsdaten deutlich übersichtlicher, schneller und personalisiert nutzbar macht. Damit verschiebt sich der Fokus von einer rein passiven Anzeige langer Listen hin zu einem aktiven, alltagsnahen Werkzeug für Schülerinnen, Schüler und Lehrkräfte.

Darüber hinaus verbindet das Thema mehrere zentrale Teilbereiche der Informatik in sinnvoller Weise: Webentwicklung, Datenverarbeitung, Schnittstellenanalyse, Rollen- und Rechtestrukturen, Persistenz, automatisierte Tests sowie die Umsetzung von Hintergrundprozessen. Dadurch entsteht nicht nur eine Benutzeroberfläche, sondern ein vollständiges Softwaresystem mit begründeten Architekturentscheidungen. Diese fachliche Breite und Tiefe macht das Projekt besonders geeignet für eine Besondere Lernleistung im Fach Informatik.

1.3 Warum eine Webanwendung?

Die Entscheidung für eine Webanwendung wurde bewusst getroffen. Eine native App hätte getrennte Entwicklungs-, Verteilungs- und Updateprozesse für iOS und Android erfordert, während eine Desktop-Anwendung der tatsächlichen Nutzungssituation nicht gerecht geworden wäre, da Vertretungspläne überwiegend mobil - etwa zwischen Unterrichtsstunden - abgerufen werden.

Eine Webanwendung bietet in diesem Kontext mehrere entscheidende Vorteile: Sie ist ohne Installation sofort erreichbar, plattformunabhängig auf Smartphone, Tablet und Desktop nutzbar und kann zentral aktualisiert werden, ohne dass ein App-Store-Prozess notwendig ist. Dadurch ist sie insbesondere in schulischen Umgebungen deutlich zugänglicher und wartungsärmer.

Gleichzeitig bleibt die Umsetzung technisch anspruchsvoll, da zentrale Aspekte wie Caching, Sicherheit, Zustandsverwaltung, Authentifizierung sowie server- und clientseitige Kommunikation eigenständig gelöst werden müssen. Auch die Integration von Push-Benachrichtigungen ist inzwischen im Webkontext möglich: Seit 2023 unterstützen alle großen Browser - einschließlich Safari auf iOS - den Web-Push-Standard[1], sodass Benachrichtigungen ohne native App realisiert werden können.

Insgesamt stellt die Webanwendung somit eine sowohl praxisnahe als auch fachlich fundierte Lösung dar, die den Anforderungen des Nutzungskontexts und der technischen Umsetzung gleichermaßen gerecht wird.

[1] vgl. W3C Push API: https://www.w3.org/TR/push-api/

1.4 Einordnung als Besondere Lernleistung

Die schriftliche Ausarbeitung orientiert sich an den Rahmenbedingungen der Besonderen Lernleistung im Landesabitur Hessen. Entscheidend ist dabei nicht nur das fertige Produkt, sondern auch die nachvollziehbare Dokumentation des Arbeitsprozesses, der Methoden, der Entscheidungen und der Reflexion. Die Arbeit bezieht sich auf ein Projekt, das über mindestens zwei Halbjahre hinweg entwickelt wurde und in einem Kolloquium verteidigt werden kann.

Daraus folgt für diese Dokumentation ein klarer Anspruch: Sie muss zeigen, warum das Thema gewählt wurde, wie Anforderungen entstanden sind, wie die technische Lösung aufgebaut wurde, welche Schwierigkeiten aufgetreten sind und wie die gesetzten Ziele bewertet werden. Die Dokumentation ist damit zugleich Produktbeschreibung, Prozessdarstellung und Reflexion.

1.5 Formale Eckpunkte der BLL

Aspekt	Bedeutung für diese Ausarbeitung
Schriftliche Dokumentation	Die Arbeit beschreibt Problem, Anforderungen, Vorgehen, Umsetzung, Ergebnisse und Reflexion in zusammenhängender Form.
Kolloquium	Die Dokumentation ist so aufgebaut, dass Entscheidungen, Methoden und Grenzen im späteren Kolloquium begründet werden können.
Prozessdarstellung	Nicht nur das Ergebnis, sondern der Weg dorthin - also Iterationen, Fehlversuche und Korrekturen - wird ausdrücklich mit dargestellt.
Eigenständigkeit	Eigene Architekturentscheidungen, Reverse-Engineering der Datenquelle und selbst entwickelte Logik stehen im Mittelpunkt.
Quellenarbeit	Verwendete technische und organisatorische Quellen werden im Quellenverzeichnis gesammelt ausgewiesen.

Für meine Dokumentation bedeutet das konkret: Die Kapitelstruktur muss sowohl die fachliche Lösung als auch meinen Arbeitsprozess sichtbar machen. Deshalb werden im weiteren Verlauf nicht nur Funktionen beschrieben, sondern auch Begründungen, Fehlversuche, Rückmeldungen und Lessons Learned.

1.6 Aufbau der Arbeit

Nach der Einleitung beschreibt Kapitel 2 die Informationsquellen und die Anforderungen an das System. Kapitel 3 übersetzt diese Anforderungen in ein Pflichtenheft und erläutert die zentrale Lösungsarchitektur. Kapitel 4 dokumentiert das iterative Vorgehen, die drei Entwicklungsversionen sowie Zeitplanung und Meilensteine. Kapitel 5 ordnet den verwendeten Tech Stack fachlich ein. Kapitel 6 zeigt das System in Diagrammen aus Nutzungs-, Struktur- und Ablaufperspektive. Kapitel 7 konzentriert sich auf die technische Umsetzung, Schwierigkeiten und Learnings. Kapitel 8 bewertet die Ergebnisse anhand der zuvor definierten Ziele, bevor Kapitel 9 ein Fazit zieht und einen Ausblick gibt.

2 Informationsquellen und Anforderungen (Lastenheft)

2.1 Woher stammen die Anforderungen?

Die Anforderungen wurden nicht aus einem fertigen Pflichtenheft übernommen, sondern aus mehreren praxisnahen Quellen zusammengeführt.[2] Die wichtigste Quelle war der schulische Alltag selbst: die wiederkehrende Erfahrung, dass vorhandene Vertretungsinformationen zwar digital vorliegen, aber zu langsam, zu unübersichtlich und zu wenig personalisiert nutzbar sind.

Hinzu kamen Gespräche mit den betreuenden Lehrkräften, eigene Notizen aus den Projekttreffen sowie Rückmeldungen aus dem Informatik-Kurs. Diese Rückmeldungen waren vor allem dort wichtig, wo reine technische Machbarkeit noch nichts über die tatsächliche Nutzbarkeit aussagt - etwa bei der mobilen Darstellung, bei der Datumsnavigation oder beim Nutzerfluss zwischen Registrierung, Onboarding und Dashboard.

Eine weitere Informationsquelle war die vorhandene WebUntis-Anzeige selbst. Durch die Beobachtung der bestehenden Nutzungssituation ließ sich relativ klar ableiten, welche Probleme die neue Anwendung lösen sollte: schnellere Orientierung, klarere Darstellung, bessere Filterung und die Möglichkeit, persönliche Relevanz überhaupt erst abzubilden.

2.2 Zielgruppen und Beteiligte

Primäre Zielgruppe der Anwendung sind Schülerinnen, Schüler und Lehrkräfte, die Vertretungsinformationen schnell für einen konkreten Tag prüfen wollen. Für diese Gruppe steht die öffentliche oder personalisierte Nutzung im Vordergrund. Eine sekundäre Rolle nehmen Administratoren ein, die Lehrerkürzel pflegen, Nutzerrollen verwalten und damit die Grundlage für die personalisierte Nutzung schaffen. Als externes System ist WebUntis zu

[2] Vgl. Sommerville (2015), Kap. 4: Anforderungserhebung aus heterogenen Quellen als etabliertes Vorgehen im Requirements Engineering.

berücksichtigen, weil dort die Datenquelle liegt, auch wenn die neue Anwendung die Daten eigenständig aufbereitet.

2.3 Funktionale Anforderungen

Aus diesen Quellen ergaben sich mehrere Kernanforderungen. Erstens muss die Anwendung Vertretungen für ein konkretes Datum abrufen und übersichtlich anzeigen. Zweitens muss sie Such- und Filtermöglichkeiten bereitstellen, damit Nutzerinnen und Nutzer relevante Einträge schneller finden. Drittens muss sie definierte Zustände für Sonderfälle bieten: kein Plan, Teildaten, Fehler bei der Datenquelle oder Sperrung durch Rate-Limits.

Darüber hinaus sollte das System auch über eine reine öffentliche Anzeige hinausgehen. Für angemeldete Nutzerinnen und Nutzer sollte ein persönlicher Stundenplan hinterlegt werden können, damit relevante Vertretungen gesondert hervorgehoben oder später per Push versendet werden können. Zusätzlich waren Rollen- und Verwaltungsfunktionen notwendig, damit Lehrerkürzel, Nutzerrechte und Einstellungen nicht direkt im Code gepflegt werden müssen.

2.4 Nicht-funktionale Anforderungen

Neben den sichtbaren Funktionen spielten mehrere qualitative Anforderungen eine wichtige Rolle. Die Anwendung sollte kurze Antwortzeiten erreichen, robust auf fehlerhafte oder unvollständige Antworten reagieren und eine klare Trennung von Zuständigkeiten im Code aufweisen. Sicherheit war ebenfalls ein Thema: geschützte Bereiche dürfen nicht ohne Authentifizierung erreichbar sein, mutierende Requests müssen abgesichert werden und die Browser-Auslieferung soll über geeignete Header gehärtet werden.

Wartbarkeit war für das Projekt ebenfalls wichtig. Weil die Anwendung über mehrere Entwicklungsphasen gewachsen ist, musste die spätere Version so strukturiert werden, dass Änderungen an einer Stelle nicht automatisch an mehreren anderen Stellen neue Fehler erzeugen. Deshalb wurden Hook, API-Route, Datenverarbeitung, Datenbankmodell und UI-Komponenten getrennt betrachtet.

2.5 Bedienung im Schulalltag

Die Bedienung sollte möglichst wenig Erklärung brauchen.[3] Für Gäste muss der Einstieg sofort funktionieren: Seite öffnen, Datum wählen, Ergebnisse lesen. Für registrierte Nutzer sollte die Anwendung stärker führen. Nach der Registrierung folgt die einmalige Einrichtung des eigenen Stundenplans, danach wird das Dashboard zum persönlichen Einstiegspunkt. Diese geführte Struktur entstand nicht aus Komfortgründen allein, sondern aus der Einsicht, dass eine

[3] Vgl. Krug (2014), Kap. 1–2: „Don't Make Me Think" als Leitsatz für selbsterklärende Interfaces.

technisch korrekte Funktion ohne verständlichen Nutzerfluss im Alltag kaum angenommen wird.

2.5A Typischer Nutzungsablauf

Ein typischer Nutzungsablauf beginnt mit dem Öffnen der Startseite im Browser. Ohne Anmeldung kann ein Gast sofort ein Datum wählen und die Vertretungsliste einsehen. Suche und Kategorienfilter stehen direkt zur Verfügung. Wer personalisierte Funktionen nutzen möchte, registriert sich mit E-Mail-Adresse und Passwort. Nach dem ersten Login folgt automatisch das Onboarding: Der eigene Stundenplan wird Schritt für Schritt eingetragen. Danach zeigt das Dashboard nur noch die für den eigenen Plan relevanten Vertretungen. Push-Benachrichtigungen lassen sich in den Einstellungen aktivieren; ab diesem Moment erhält der Nutzer automatisch eine Nachricht, sobald sich relevante Vertretungen ändern. Administratoren finden im Admin-Bereich zusätzlich die Lehrerkürzel-Pflege und die Nutzerverwaltung.[4]

2.6 SMART-Ziele[5]

ID	Spezifisch	Messbar	Relevant	Terminiert
S1	Vertretungsinfos für ein Datum in höchstens drei Interaktionen erreichen.	Datum wählen, laden, Ergebnis sichtbar; kein zusätzlicher Seitenwechsel.	Spart Zeit im Schulalltag.	30.11.2025
S2	Such- und Filterfunktionen für Klassen, Fächer und Typen bereitstellen.	Textsuche plus Kategorienfilter funktionieren stabil.	Erhöht die Orientierung bei vielen Einträgen.	15.12.2025
S3	Robuste API-Nutzung mit klaren	Rate-Limit, Retry-Logik und definierte Fehlerantworten sind implementiert und getestet.	Verringert Ausfälle und Frust.	22.12.2025

[4]Vgl. Krug (2014), Kap. 3–4: Konventionen und klare visuelle Hierarchie als Grundlage für intuitive Bedienung.
[5]Vgl. Sommerville (2015), Kap. 4: SMART-Kriterien als Grundlage messbarer Zielformulierung.

ID	Spezifisch	Messbar	Relevant	Terminiert
	Fehlerpfaden sicherstellen.			
S4	Sicherheits- und Datenschutzbasis für den Browserbetrieb schaffen.	CSP aktiv, geschützte Bereiche abgesichert, Pflichtseiten erreichbar.	Erfüllt Web- und Schulgrundanforderungen.	10.01.2026
S5	Wartbarkeit und Nachvollziehbarkeit der Lösung absichern.	Kernrouten, Hook und zentrale Logik sind dokumentiert und getestet.	Erleichtert spätere Änderungen.	31.01.2026

2.7 Zentrale Anwendungsfälle

1. Ein Gast ruft die Startseite auf, wählt ein Datum und erhält direkt den Vertretungsplan für diesen Tag. Suche und Kategorienfilter können die Liste weiter eingrenzen. Für diesen Kernfall ist kein Login notwendig.

2. Ein neuer Nutzer registriert sich mit E-Mail-Adresse und Passwort. Danach folgt einmalig das Onboarding: Der persönliche Stundenplan wird eingetragen, damit das Dashboard später relevante Vertretungen erkennen kann.

3. Ein angemeldeter Nutzer öffnet das Dashboard und sieht dort die Vertretungen des gewählten Tages bereits im Bezug zum eigenen Stundenplan. Relevante Einträge können dadurch schneller erkannt werden.

4. Ein Nutzer mit aktiven Push-Benachrichtigungen erhält nur dann eine Nachricht, wenn sich relevante Vertretungen neu ergeben oder geändert haben. Unveränderte Daten sollen bewusst keine zusätzliche Nachricht auslösen.

5. Ein Administrator pflegt Lehrerkürzel, verwaltet Rollen und schützt administrative Änderungen durch geeignete Bestätigungen. Damit wird aus einer reinen Anzeige ein betreibbares Webprodukt.

3 Lösungsansatz und Pflichtenheft

3.1 Grundidee der Lösung

Die Grundidee des Projekts besteht darin, zwischen Nutzer und Datenquelle eine eigene Anwendungsschicht zu setzen. Diese neue Schicht soll drei Dinge leisten: erstens die vorhandenen Vertretungsdaten verständlicher aufbereiten, zweitens persönliche Relevanz abbilden und drittens Fehler, Lastspitzen und technische Sonderfälle kontrolliert behandeln. Das Ziel ist also nicht, WebUntis zu ersetzen, sondern die Nutzung der vorhandenen Daten für den Alltag der Schule deutlich zu verbessern.

3.2 Architekturprinzip

Die finale Architektur trennt vier Ebenen. Auf der ersten Ebene steht die Benutzeroberfläche mit Startseite, Dashboard und Einstellungen. Die zweite Ebene bildet der Client-Zustand im Browser, insbesondere der Hook für den Datenabruf. Die dritte Ebene ist die serverseitige API-Schicht, die Eingaben validiert, Fehlerbehandlung bündelt, Rate-Limits schützt und den Server-Cache verwaltet. Die vierte Ebene ist die externe Datenquelle WebUntis.[6]

Diese Trennung war für die Wartbarkeit entscheidend. In den frühen Versionen war zu viel Logik direkt in der Oberfläche untergebracht. Erst mit einer sauberen Aufteilung ließ sich erreichen, dass neue Funktionen - etwa Push-Benachrichtigungen oder Rollenlogik - nicht ständig an mehreren Stellen gleichzeitig nachgerüstet werden mussten.

3.3 Öffentliche Nutzung, Registrierung und Dashboard

Die Anwendung ist bewusst zweistufig angelegt. Der erste Zugriff funktioniert öffentlich, weil die Grundfunktion - Vertretungen ansehen - möglichst niedrigschwellig sein soll. Die personalisierten Funktionen wie Stundenplanabgleich oder Push-Benachrichtigungen setzen dagegen einen Login voraus. Dadurch bleibt die Anwendung für spontane Nutzung offen, ohne auf erweiterte Funktionen verzichten zu müssen.

Nach der Registrierung folgt deshalb ein geführter Ablauf: zunächst der Login, danach die einmalige Einrichtung persönlicher Daten und erst dann der Übergang ins Dashboard. Für Administratoren kommt zusätzlich ein Setup-Pfad hinzu, damit Lehrerkürzel und Rollen nicht manuell im Code verändert werden müssen. Dieses Onboarding war ein zentraler Teil des Pflichtenhefts, weil es die technische Lösung erst alltagstauglich macht.

---

[6] *Vgl. Sommerville (2015), Kap. 6: Schichtenarchitektur als Mittel zur Trennung von Zuständigkeiten in Softwaresystemen*

3.4 Rollen und Berechtigungen

Das Rollenkonzept ist bewusst klein gehalten. Es gibt normale Nutzer mit Zugriff auf persönliche Funktionen sowie Administratoren mit erweiterten Rechten für Lehrerkürzel, Rollenverwaltung und betreiberspezifische Einstellungen. Der erste Administrator wird über eine Umgebungsvariable definiert. Danach können weitere Rollen in der Weboberfläche vergeben werden. Das Herabstufen eines Administrators ist zusätzlich abgesichert, damit nicht versehentlich der letzte vorhandene Administrator entfernt wird.

3.5 Datenmodell und Entitäten

Das relationale Datenmodell bildet die Grundlage für alle personalisierten Funktionen der Anwendung. Im Zentrum steht der Nutzer mit seiner Rolle und seinen Zugangsdaten. Jeder angemeldete Nutzer kann Stundenplaneinträge anlegen, die Wochentag, Stunde, Fach, Lehrkraft und Wochenmodus enthalten. Diese Einträge sind die Basis für den personalisierten Abgleich: Beim Push-Dispatch werden die aktuellen Vertretungsdaten mit dem hinterlegten Stundenplan verglichen.

Zur Unterstützung der Eingabe merkt sich das System häufig verwendete Fach-Lehrkraft-Kombinationen als Vorschläge. Diese sogenannten Stundenplan-Presets besitzen bewusst keinen Zeitbezug und sind damit unabhängig vom eigentlichen Stundenplaneintrag.

Für die Benachrichtigungslogik existieren zwei getrennte Tabellen: Der Benachrichtigungszustand speichert pro Nutzer und Tag genau einen Fingerprint und wird bei jeder Prüfung aktualisiert oder gelöscht. Der Benachrichtigungsabdruck protokolliert dagegen jeden gesendeten Fingerprint als unveränderliches Audit-Log. Diese Trennung ermöglicht sowohl die Delta-Entscheidung (senden, überspringen oder auflösen) als auch die Nachvollziehbarkeit vergangener Benachrichtigungen.

Ergänzend verwaltet das Lehrerverzeichnis die Zuordnung von Kürzeln zu Klarnamen, und die Anwendungseinstellungen steuern betreiberspezifische Konfiguration wie erlaubte E-Mail-Domänen.

3.6 Datenbeschaffung über WebUntis

Für die Datenbeschaffung musste zunächst verstanden werden, wie die bestehende WebUntis-Ansicht ihre Informationen lädt. Da keine fertige öffentliche Dokumentation für diesen Anwendungsfall vorlag, wurde die Schnittstelle durch Browser-Netzwerkanalyse

erschlossen. Ich habe untersucht, welche Requests bei der Anzeige ausgelöst werden, wie Datumswerte codiert sind und welche Antwortstrukturen zurückkommen.

Dieses Vorgehen war eine typische informatische Analyseaufgabe: Hypothesen bilden, Parameter variieren, Antworten prüfen und daraus ein belastbares Modell ableiten. Die Anwendung spricht WebUntis deshalb nicht direkt aus dem Browser an, sondern immer über die eigene Server-Schicht. Dadurch bleiben Validierung, Fehlerbehandlung und Begrenzung der Last in meiner Kontrolle.

3.7 API-Proxy, Caching und Fehlerpfade

Der API-Proxy ist einer der wichtigsten Bausteine des Projekts. Er nimmt Requests aus dem Frontend entgegen, validiert die Datumsangaben, begrenzt die Request-Frequenz und kapselt den Zugriff auf WebUntis. Wenn ein Nutzer oder mehrere Nutzer in kurzer Zeit dieselben Daten abfragen, sollen nicht jedes Mal neue Upstream-Requests entstehen. Deshalb gibt es einen Server-Cache, ergänzt durch einen Client-Cache im Hook.[7]

Ebenso wichtig ist die Unterscheidung verschiedener Antworttypen. Es muss einen klaren Unterschied geben zwischen einem Tag ohne Vertretungen, einer Antwort mit unvollständigen Metadaten und einem echten Fehlerfall. Nur so kann die Oberfläche sinnvoll auf die jeweilige Situation reagieren. Gerade diese Fehlerpfade waren notwendig, um aus einer reinen Demo ein belastbares Webprodukt zu machen.

3.8 Push-Benachrichtigungen und Delta-Logik

Push-Benachrichtigungen wurden nicht als einfache 'Sende immer den aktuellen Stand'-Funktion umgesetzt. Eine solche Lösung wäre zwar technisch leicht, hätte im Alltag aber kaum Nutzen, weil Nutzer bei jedem Abgleich erneut dieselben Informationen bekämen. Stattdessen wird bei jedem Prüflauf ein Fingerprint über die relevanten Vertretungen eines Nutzers gebildet.

Nur wenn sich dieser Fingerprint im Vergleich zum letzten gespeicherten Zustand verändert, wird eine neue Push-Nachricht gesendet. Dadurch wird das System sparsamer, relevanter und benutzerfreundlicher. Gleichzeitig bleibt nachvollziehbar, welche Benachrichtigungen neu, unverändert oder aufzulösen sind.[8]

Für die Zustellung im Browser ist ein Service Worker registriert. Er empfängt Push-Events auch bei geschlossenem Tab und zeigt die Benachrichtigung über die Notification API des

[7] Vgl. RFC 5861 – HTTP Cache-Control Extensions for Stale Content: Grundlage für das stale- while-revalidate-Muster, das hier adaptiert wurde.
[8] Die technische Grundlage für Web Push bildet RFC 8292 – Voluntary Application Server Identification (VAPID), das die Authentifizierung des Push-Servers gegenüber dem Browser regelt.

Browsers an. Zusätzlich behandelt er Klick-Events auf die Benachrichtigung, um den Nutzer direkt zur relevanten Tagesansicht zu führen.[9]

3.9 Warum 15-Minuten-Polling statt Echtzeit?

Die Entscheidung für ein 15-Minuten-Intervall ist kein zufälliger Kompromiss, sondern das Ergebnis mehrerer Abwägungen. WebUntis stellt keinen dauerhaften Event-Stream bereit, an den sich die Anwendung anhängen könnte. Echte Sekunden-Echtzeit würde deshalb häufiges Polling bedeuten. Das würde sowohl die externe Datenquelle stärker belasten als auch die Fehleranfälligkeit erhöhen.

Für den Schulalltag bringt Sekunden-Echtzeit außerdem nur einen geringen Mehrwert. Entscheidend ist nicht, ob eine Vertretung nach drei oder nach dreißig Sekunden sichtbar wird, sondern ob Änderungen verlässlich, geordnet und ohne Benachrichtigungslärm ankommen. Unter diesen Bedingungen ist ein Intervall von 15 Minuten die vernünftige technische und praktische Lösung.

3.10 Hosting und Bereitstellung

Damit die Anwendung nicht nur lokal funktioniert, sondern als echtes Webprodukt nutzbar ist, muss sie unter einer öffentlichen HTTPS-Adresse erreichbar sein. Das ist insbesondere für den externen Scheduler QStash relevant, weil dieser keine lokalen Endpunkte aufrufen kann. Gleichzeitig setzt der produktive Betrieb korrekt konfigurierte Umgebungsvariablen, abgesicherte Routen und einen reproduzierbaren Deploy-Prozess voraus.

Die Hosting-Entscheidung gehört deshalb fachlich zum Pflichtenheft dazu. Erst durch den öffentlichen Betrieb werden Themen wie SSL, Secrets, Scheduler, Browserrechte und Push-Zustellung praktisch relevant. Das Projekt endet also nicht beim funktionierenden lokalen Code, sondern erst beim belastbaren Betrieb im Web.

4 Projektmanagement und Zeitplanung

4.1 Iteratives Vorgehen statt starrem Vorgehensmodell

Für dieses Projekt war kein klassisches Wasserfallmodell sinnvoll. Zu Beginn stand zwar die Grundidee fest, aber viele Details - etwa Rollenmodell, Push-Strategie oder die genaue Trennung zwischen öffentlicher Nutzung und persönlichem Dashboard - waren noch nicht

[9] Vgl. W3C Service Workers Spezifikation.

vollständig geklärt. Deshalb wurde iterativ gearbeitet: ausprobieren, testen, Schwächen erkennen, Architektur verbessern und den nächsten Meilenstein ableiten.[10]

Dieses Vorgehen war nicht Ausdruck von Planlosigkeit, sondern eine passende Methode für ein Einzelprojekt mit realem Produktbezug. Gerade weil die Anwendung im Schulalltag erprobt werden konnte, war es sinnvoll, mehrere Zwischenstände zu entwickeln und anhand echter Rückmeldungen weiterzuarbeiten.

4.2 Entwicklungsversionen

Die formale Themeneinreichung erfolgte am 31. Januar 2025. Die eigentliche technische Umsetzung begann im August 2025 mit einer sehr einfachen ersten Version. Diese frühe Fassung war eine statische Website ohne Backend. Die Datumsauswahl wurde über URL-Parameter gesteuert, die Daten kamen direkt im Browser von der WebUntis-Schnittstelle. Diese Version zeigte schnell ihre Grenzen: Alle Vertretungen wurden ungefiltert angezeigt, eine Personalisierung war damit nicht möglich.

Die zweite Version setzte bereits auf Next.js, blieb aber noch eine überwiegend frontendlastige Anwendung ohne saubere API-Schicht und ohne Authentifizierung. Mit wachsendem Umfang wurde deutlich, dass dieser Ansatz nicht stabil weitergeführt werden kann. Neue Anforderungen destabilisierten vorhandenen Code zu leicht, und die Trennung von Darstellung, Datenverarbeitung und Zustandslogik war nicht klar genug.

Die dritte Version war deshalb ein bewusster Neustart. Erst wurde das Datenmodell sauber gedacht, dann die API-Schicht aufgebaut und erst danach die Oberfläche darauf abgestimmt. Authentifizierung, Rollen, Caching und Push-Benachrichtigungen wurden von Beginn an als Systembestandteile mitgeplant. Diese Version ist die Grundlage der vorliegenden schriftlichen Ausarbeitung und der produktiv genutzten Anwendung.

4.3 Meilensteine4.4 Gantt-Plan

M	Fokus	Ausgangsproblem	Ergebnis
M1	Grundidee und MVP	Die Problemstellung war klar, aber der Mehrwert gegenüber der Ausgangsanzeige musste erst sichtbar werden.	Öffentliche Startseite, Datumswahl und erster funktionsfähiger Prototyp.

[10] *Vgl. Sommerville (2015), Kap. 2: Iterative und inkrementelle Entwicklung als Alternative zu plangetriebenen Modellen.*

M	Fokus	Ausgangsproblem	Ergebnis
M2	API und Datenrobustheit	Der Abruf externer Daten musste trotz Upstream-Schwankungen verlässlich funktionieren.	API-Proxy, Validierung, Retry-Logik, Rate-Limit und definierte Fehlerpfade.
M3	Web-Bereitstellung und Nutzerfluss	Aus einer Anzeige musste ein nutzbares Produkt mit Konto, Rollen und Dashboard werden.	Registrierung, Admin-Setup, Onboarding und persönliches Dashboard.
M4	Push-Benachrichtigungen	Relevante Änderungen sollten gemeldet werden, ohne Nutzer zu spammen.	QStash-Dispatch, Delta-Logik und Push-Tests auf mehreren Geräten.
M5	Dokumentation und Abgabe	Der Projektstand musste formal sauber und nachvollziehbar beschrieben werden.	Kapitelstruktur, Diagramme, Zeitplan, Testnachweise und Ergebnisbewertung.

Der zeitliche Verlauf des Projekts lässt sich in einem Gantt-Plan besonders gut darstellen. Sichtbar wird dabei, dass nicht alle Themen parallel bearbeitet werden konnten. Vor allem Push-Benachrichtigungen und produktives Hosting wurden erst sinnvoll, nachdem API-Stabilität und Nutzerfluss bereits gelöst waren.

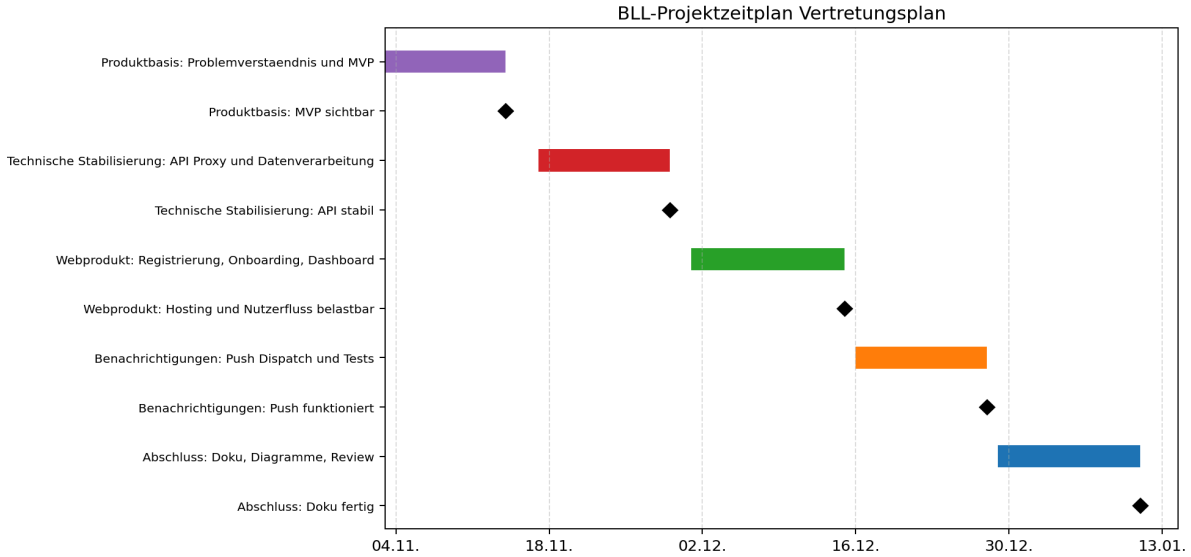

Abbildung 1: Gantt-Diagramm des Projektverlaufs mit zentralen Meilensteinen

4.5 Netzplan und kritischer Pfad

Neben der zeitlichen Planung ist auch die Abhängigkeit zwischen den Arbeitsschritten wichtig. Der kritische Pfad verläuft von der Schärfung der Anforderungen über MVP, Datenabruf, API-Stabilisierung und Nutzerfluss bis zur Push-Integration und schließlich zur Dokumentation. Einzelne Teilaufgaben wie zusätzliche Filterlogik oder Benutzertests konnten zwar parallel laufen, hingen aber inhaltlich weiterhin an den Kernschritten.

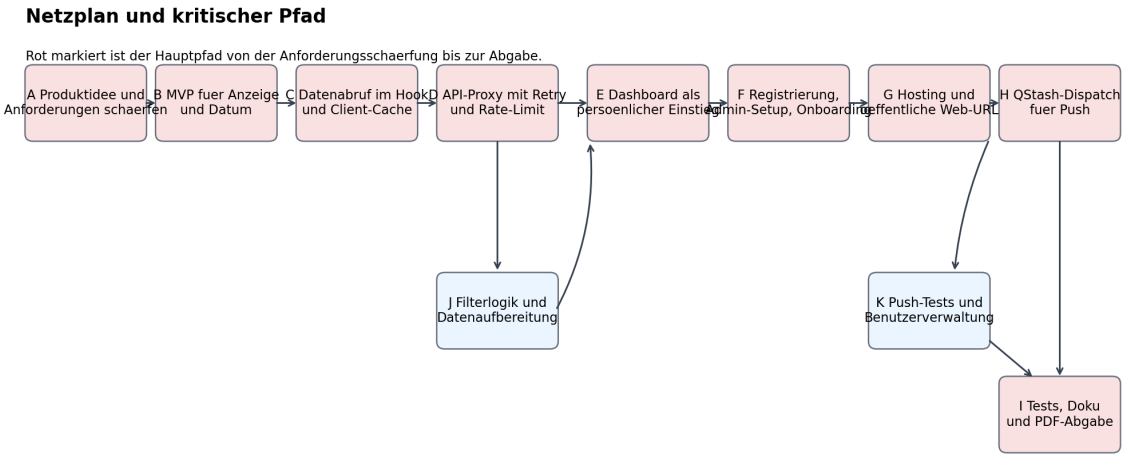

Abbildung 2: Netzplan mit kritischem Pfad und ergänzenden Teilsträngen

4.6 Rückmeldungen und Anpassungen

Die Entwicklung wurde durch mehrere Rückmeldungsschleifen begleitet. Über das Projekt hinweg gab es mündliche Rückmeldungen durch die betreuenden Lehrkräfte Alexander Rhode und Michael Knobl. In der späteren Phase bezogen sich diese Gespräche besonders auf die Kapitelstruktur, auf fehlende inhaltliche Punkte und auf die Schärfung der schriftlichen Ausarbeitung.

Parallel dazu wurde die Anwendung von Mitschülerinnen und Mitschülern auf unterschiedlichen Geräten getestet. Gerade auf Smartphones zeigten sich praktische Probleme: abgeschnittene Inhalte, verschobene Layouts und unklare Schaltflächen. Außerdem kamen neue Funktionswünsche hinzu, unter anderem ein Dark Mode und eine bessere Datumsnavigation. Diese Rückmeldungen flossen nicht nur kosmetisch ein, sondern veränderten teilweise auch die Prioritäten der weiteren Entwicklung.

4.7 Methodische Reflexion

Rückblickend war das iterative Vorgehen für dieses Projekt die richtige Entscheidung. Die drei Versionen zeigen, dass gute Architektur nicht sofort entsteht, sondern schrittweise aus realen Anforderungen, Fehlversuchen und Korrekturen herauswächst. Gerade die frühen Prototypen waren deshalb nicht überflüssig, sondern methodisch wichtig, weil sie Anforderungen sichtbar gemacht haben, die im ersten Entwurf noch nicht scharf genug formuliert waren.

5 Tools und Tech Stack

5.1 Auswahlkriterien

Die Wahl der Werkzeuge folgte nicht dem Ziel, möglichst viele moderne Technologien zu sammeln, sondern dem Anspruch, ein stabiles und wartbares Webprodukt umzusetzen. Wichtige Kriterien waren dabei eine schnelle Iteration in der Entwicklung, gute Testbarkeit, saubere Trennung zwischen Client und Server, produktive Einsetzbarkeit und meine eigene Vorerfahrung mit dem jeweiligen Werkzeug.

5.2 Kerntechnologien

Bereich	Werkzeug	Begründung
Frontend / Full-Stack	Next.js (App Router)	Ein Projekt für UI, Routing und interne API-Routen; gute Basis für ein zusammenhängendes Webprodukt.

Bereich	Werkzeug	Begründung
UI-Logik	React + TypeScript	Komponentenbasierte Oberfläche mit klar typisierten Datenstrukturen und besserer Wartbarkeit.
Styling	Tailwind CSS	Schnelle, konsistente Gestaltung mit gut kontrollierbaren Responsivitätsregeln.
Datenbank	PostgreSQL	Relationale Persistenz für Nutzer, Stundenpläne, Push-Zustände und Einstellungen.
ORM	Prisma	Strukturiertes Datenmodell, saubere Migrationen und typsichere Datenzugriffe.
Authentifizierung	NextAuth / Auth-Layer	Geschützte Bereiche, Sitzungen und saubere Trennung zwischen öffentlicher und personalisierter Nutzung.
Scheduler	Upstash QStash	Externer Trigger für den periodischen Push-Dispatch einer Webanwendung.
Push	Web Push / VAPID[11]	Browserbasierte Benachrichtigungen ohne native App.
Tests	Vitest + Testing Library	Schnelle Unit-, Integrations- und Komponententests mit gutem Feedback in der Entwicklung.
Analytik	PostHog	Optionale, datenschutzfreundliche Nutzungsanalyse zur Identifikation häufiger Fehler und Nutzungsmuster.

5.3 Warum Framework statt reinem HTML / CSS?

Reines HTML, CSS und etwas JavaScript hätten für eine einfache statische Anzeige gereicht, nicht aber für dieses Projekt in seiner endgültigen Form. Sobald Routing, Authentifizierung,

[11] Vgl. W3C Service Workers Spezifikation: Der Service Worker empfängt Push-Events im Hintergrund und zeigt Benachrichtigungen auch bei geschlossenem Browser an.

Rollen, Caching, API-Routen, sichere Serverlogik und Hintergrundprozesse Teil des Systems werden, ist ein durchgängiges Framework deutlich sinnvoller. Next.js bündelt genau diese Querschnittsthemen in einer Form, die für ein Schulprojekt professionell genug und gleichzeitig praktisch beherrschbar bleibt.

5.4 Passung des Stacks zum Projekt

Der gewählte Stack passt deshalb gut zum Projekt, weil er schnelle Iterationen zulässt und gleichzeitig strukturiert genug für spätere Erweiterungen ist. Besonders hilfreich war die Kombination aus TypeScript, Prisma und Tests: Dadurch ließen sich Datenstrukturen, API-Verträge und Zustandslogik früh absichern. Für eine BLL im Fach Informatik ist das wichtig, weil damit sichtbar wird, dass nicht nur 'eine hübsche Website' entstanden ist, sondern eine technisch nachvollziehbare, modular aufgebaute Anwendung.[12]

[12]Vgl. Sommerville (2015), Kap. 25: Softwareentwicklung und Wartbarkeit als langfristiges Qualitätsmerkmal.

6 Systemstruktur, UML und Abläufe

Die folgenden Diagramme zeigen das Projekt aus verschiedenen Perspektiven. Gemeinsam machen sie sichtbar, dass die Anwendung nicht nur aus einzelnen Seiten besteht, sondern aus aufeinander abgestimmten Bausteinen mit klaren Zuständigkeiten.

6.1 Use-Case-Diagramm

Das Use-Case-Diagramm beschreibt die Anwendung aus Sicht der beteiligten Akteure. Sichtbar wird, dass die alltägliche Nutzung durch Schülerinnen, Schüler und Lehrkräfte im Mittelpunkt steht, während WebUntis als externe Datenquelle und PostHog optional für Analysezwecke angebunden ist.

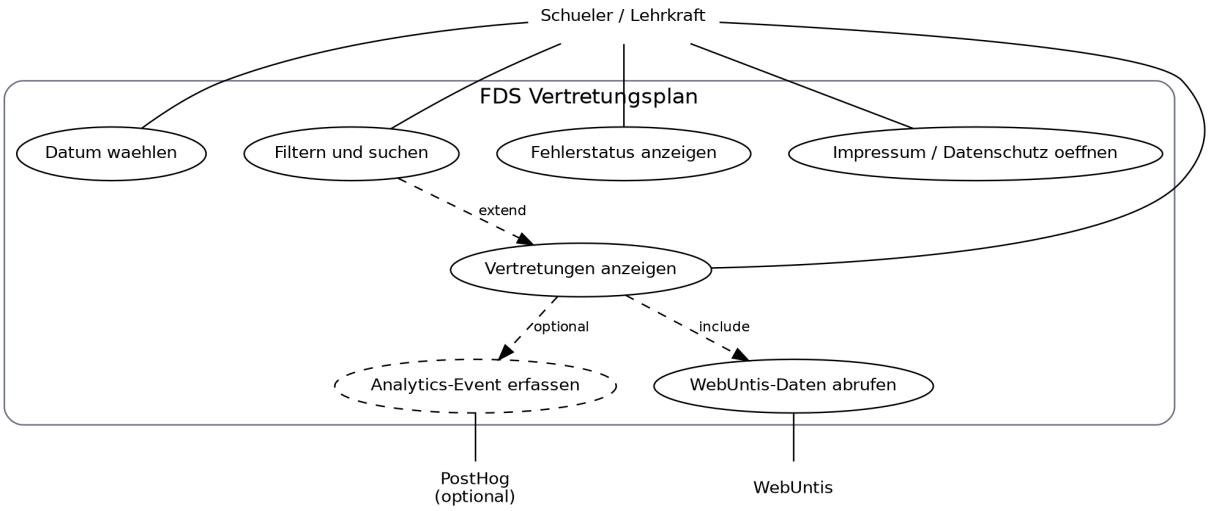

Abbildung 3: Use-Case-Diagramm für Nutzung, Datenabruf und optionale Analytik

6.2 Modulorientiertes Klassendiagramm

Im Klassendiagramm geht es nicht um Datenbankentitäten, sondern um die wichtigsten technischen Module der Anwendung. Gut erkennbar ist die Schichtentrennung: UI-Komponenten sprechen mit Hooks, Hooks sprechen mit API-Routen, und nur die serverseitigen Bereiche greifen auf externe Systeme oder die Datenbank zu. Dadurch bleibt der Browser leichtgewichtig und sicherheitskritische Logik konzentriert auf dem Server.

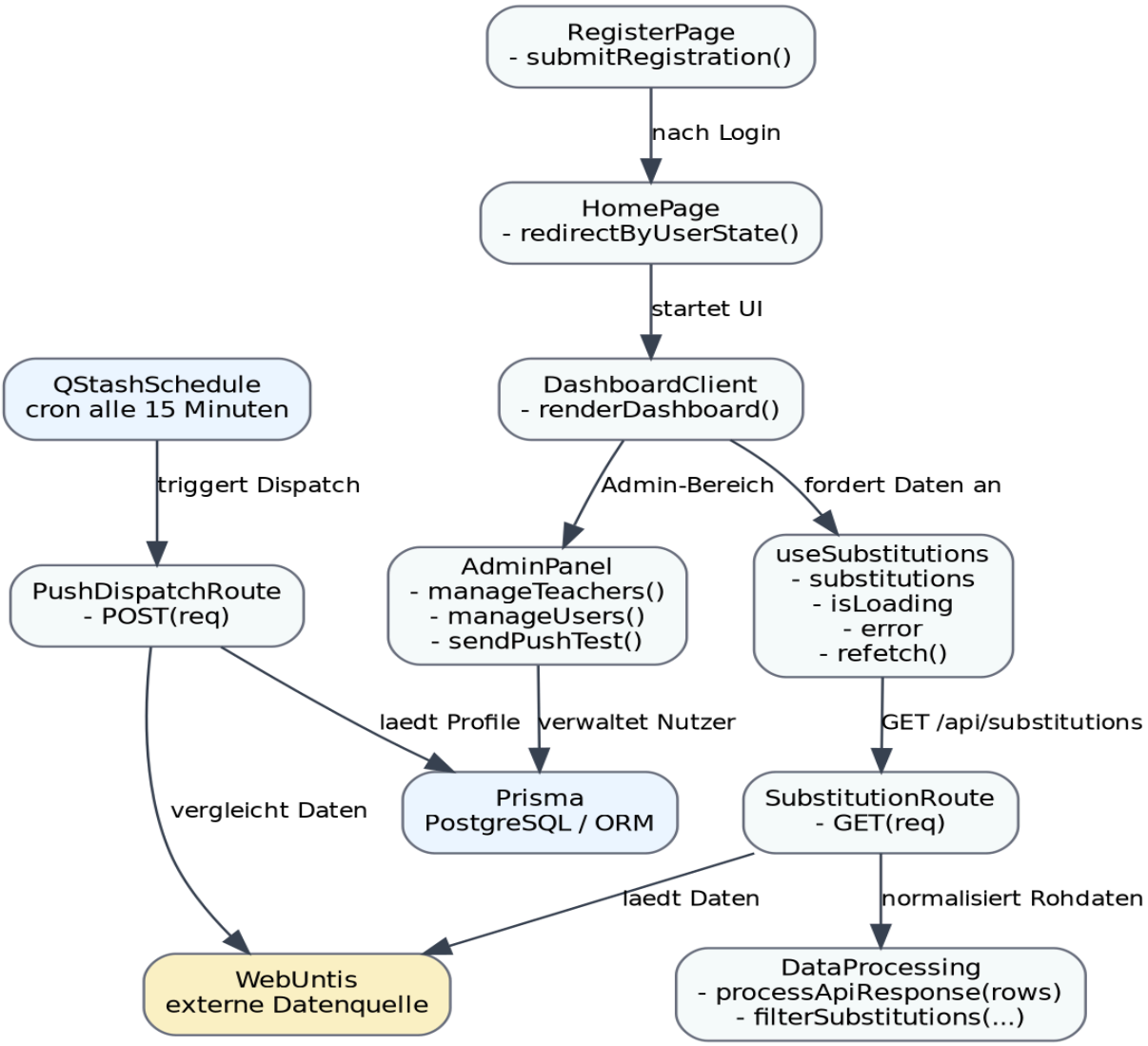

Abbildung 4: Modulorientiertes Klassendiagramm der Hauptbausteine

6.3 Objektdiagramm

Das Objektdiagramm zeigt keinen allgemeinen Modultyp, sondern einen konkreten Laufzeitzustand. Sichtbar sind etwa eine aktive Sitzung, ein persönliches Dashboard, ein konkreter Cache-Key für den Abruf und parallel dazu der Hintergrundpfad über QStash und die Push-API. Gerade für die Erklärung im Kolloquium ist diese Sicht hilfreich, weil hier abstrakte Strukturentscheidungen in eine anschauliche Betriebssituation übersetzt werden.

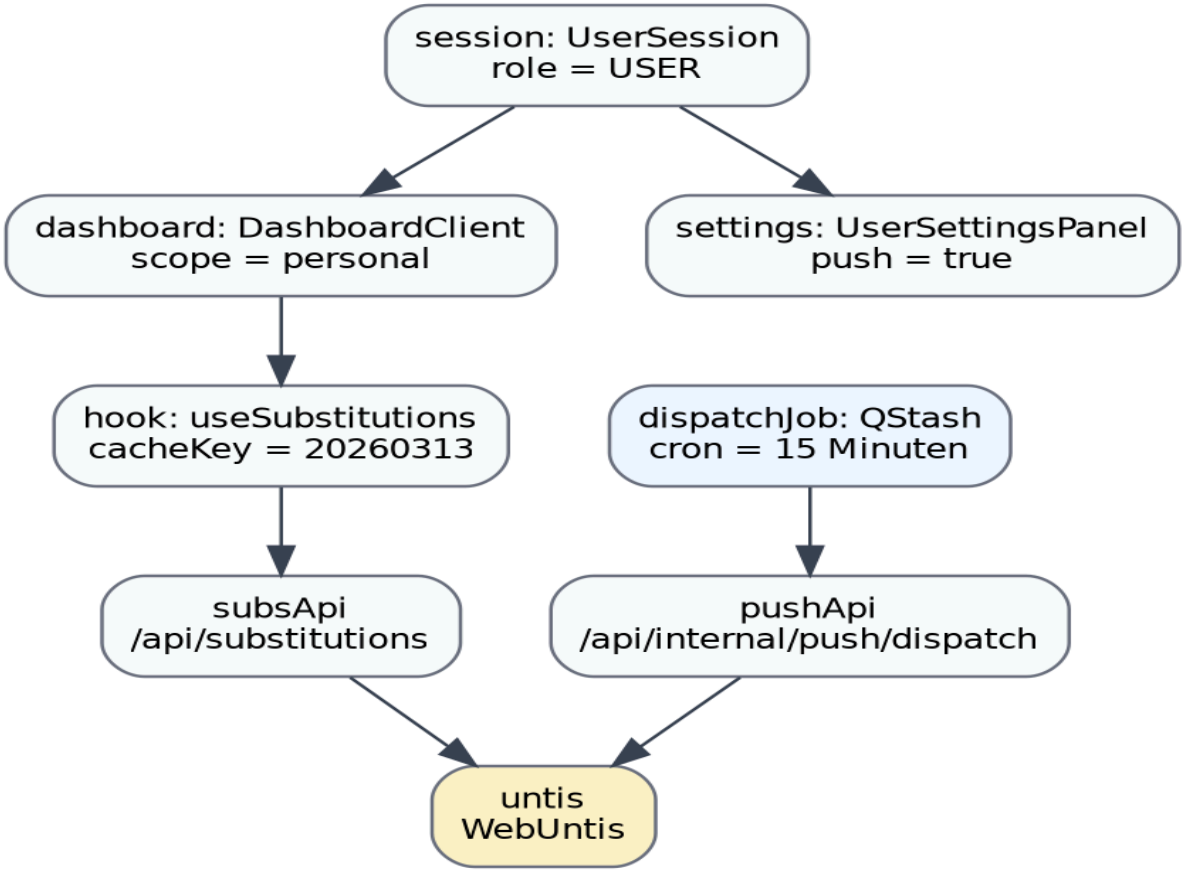

Abbildung 5: Objektdiagramm mit beispielhaftem Laufzeitzustand

6.4 Aktivitätsdiagramm: Abruf der Vertretungen

Das Aktivitätsdiagramm konzentriert sich auf die wichtigste Nutzeraktion: ein Datum auswählen und die Vertretungen abrufen. Das Diagramm macht deutlich, dass dabei mehrere Kontrollpunkte durchlaufen werden - Client-Cache, API-Proxy, Rate-Limit, Upstream-Antwort und unterschiedliche Ergebniszustände. Gerade diese Zwischenschritte unterscheiden eine robuste Anwendung von einer einfachen Direktabfrage.

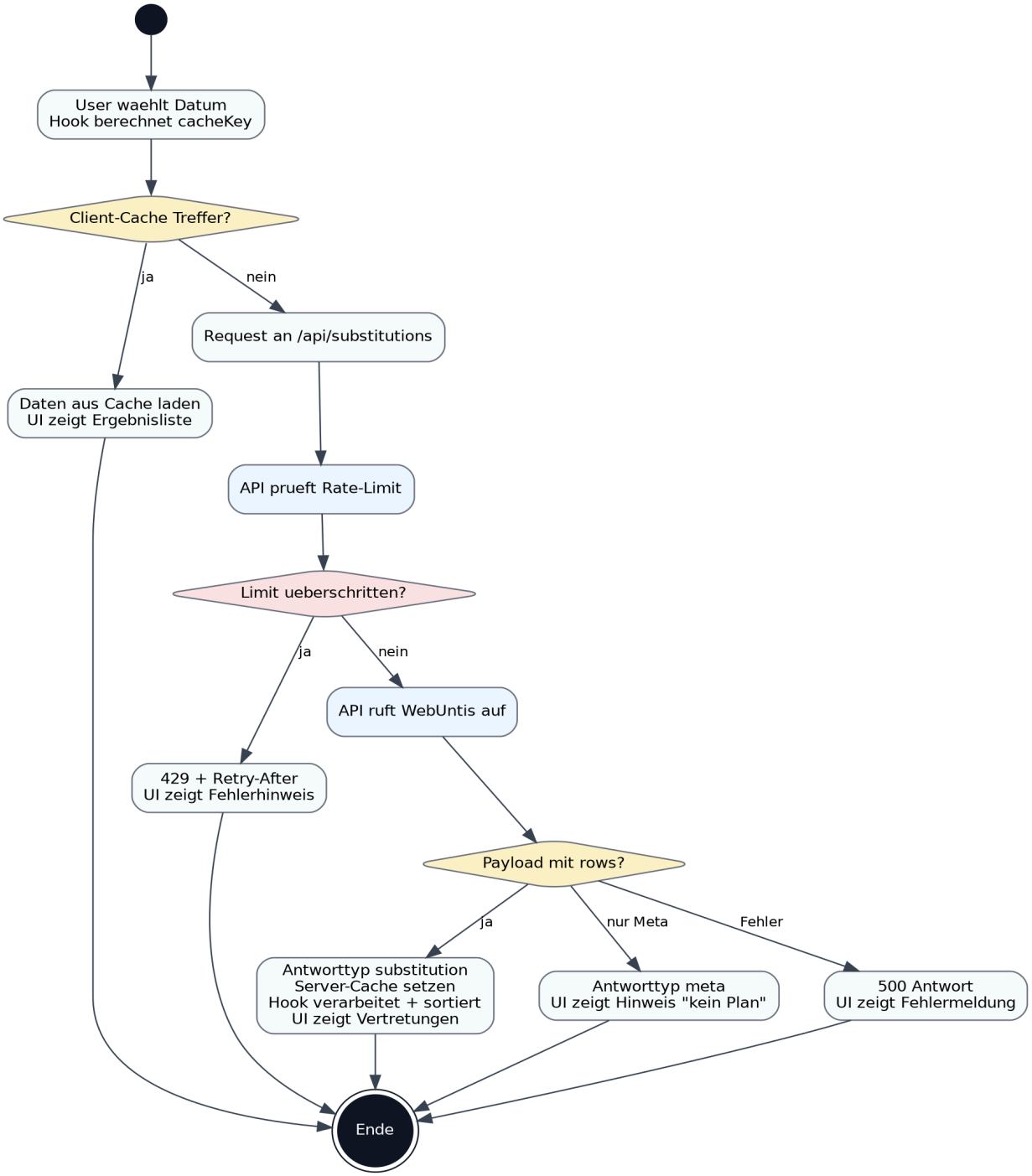

Abbildung 6: Aktivitätsdiagramm für den Abruf der Vertretungsdaten

6.5 Sequenzdiagramm: Nutzerabruf

Das erste Sequenzdiagramm beschreibt die zeitliche Abfolge zwischen Nutzeroberfläche, Hook, API und WebUntis beim normalen Abruf eines Tagesplans. Es zeigt sowohl den schnellen Pfad bei einem Cache-Treffer als auch die alternativen Fehler- und Sonderpfade. Für die Dokumentation ist diese Darstellung wichtig, weil sie die Verantwortung jedes Bausteins in zeitlicher Reihenfolge sichtbar macht.

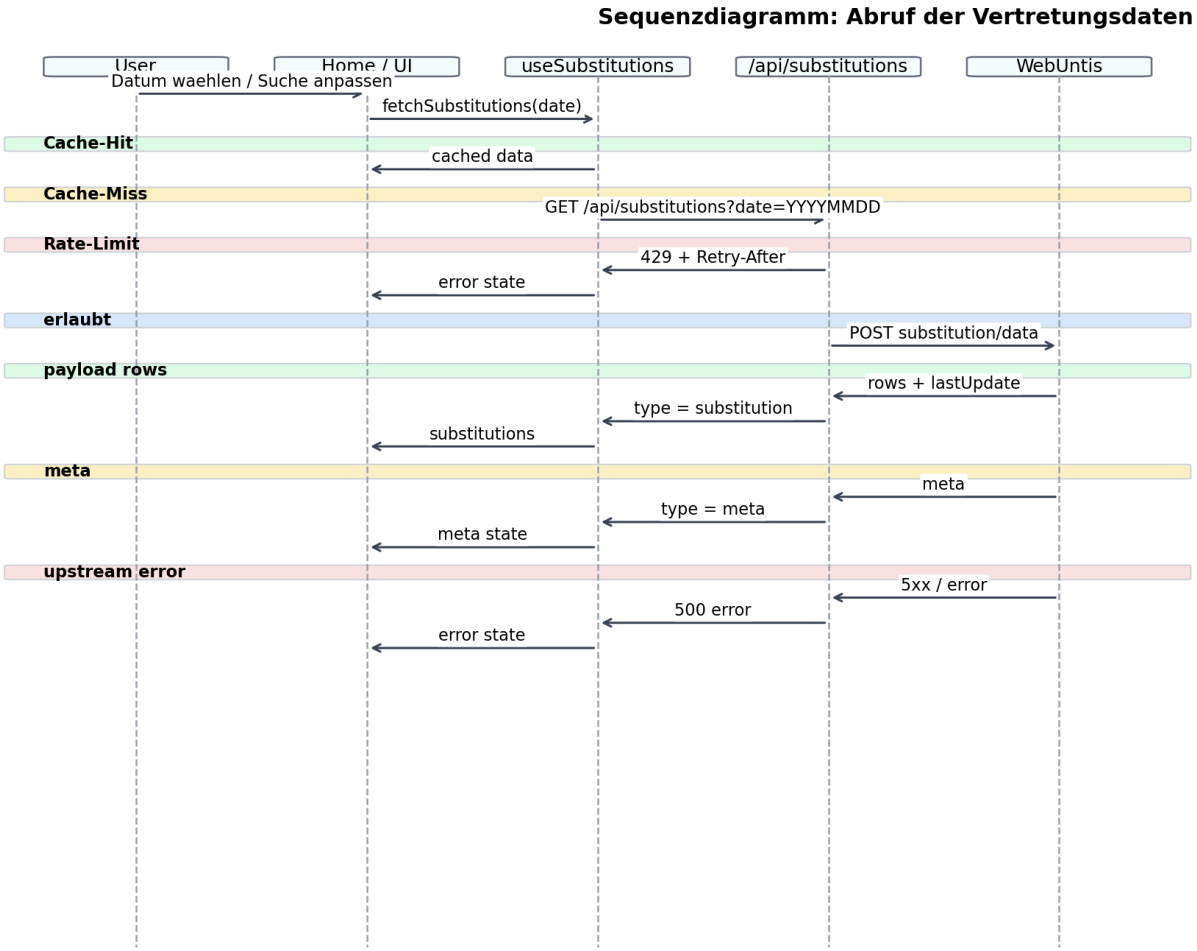

Abbildung 7: Sequenzdiagramm des regulären Nutzerabrufs

6.6 Sequenzdiagramm: Push-Dispatch

Das zweite Sequenzdiagramm behandelt den technisch anspruchsvolleren Hintergrundprozess. Hier wird deutlich, dass Push-Benachrichtigungen nicht direkt beim Laden der Seite entstehen, sondern durch einen geplanten Dispatch, der Nutzerprofile, Stundenpläne, Vertretungsdaten und gespeicherte Zustände zusammenführt. Die Darstellung macht außerdem die drei möglichen Ergebnisse sichtbar: senden, überspringen oder auflösen.

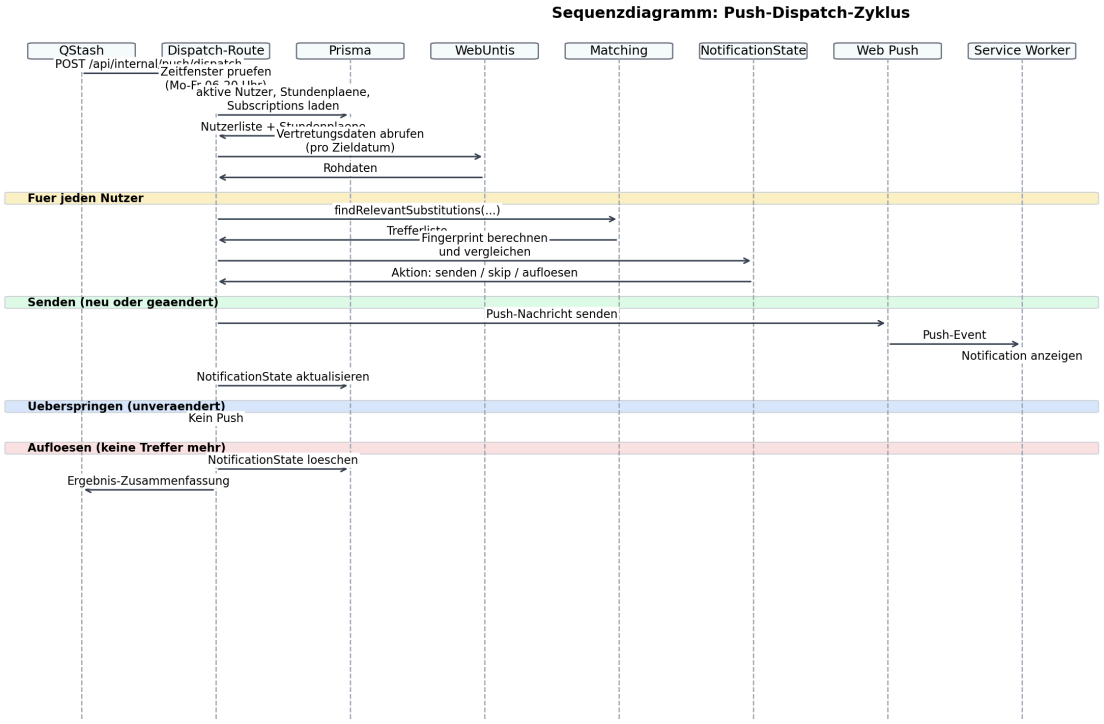

Abbildung 8: Sequenzdiagramm des Push-Dispatch-Zyklus

6.7 Bedeutung der Diagramme für die Arbeit

Die Diagramme sind nicht bloß Dekoration. Sie helfen, die Eigenleistung des Projekts nachvollziehbar zu machen. Besonders im Zusammenspiel zeigen sie, wie aus einer zunächst einfachen Idee - einen Vertretungsplan besser darzustellen - ein mehrschichtiges System mit Rollen, Hintergrundprozessen, Caching und Zustandslogik geworden ist.

7 Technische Umsetzung und Schwierigkeiten

7.1 Aufbau des Repositories

Das Repository ist nach Verantwortlichkeiten gegliedert. Diese Struktur war notwendig, um aus den frühen, eher unübersichtlichen Prototypen eine langfristig wartbare Anwendung zu entwickeln. Oberfläche, Hooks, API-Routen, Datenverarbeitung, Typdefinitionen, Provider und Datenbankmodell sind getrennt abgelegt. Dadurch wird schneller sichtbar, an welcher Stelle eine Änderung sinnvoll ist und welche Bereiche davon betroffen sind.

7.2 Zentrale Module und ihre Aufgaben

Datei / Ordner	Aufgabe im Gesamtsystem
src/app/page.tsx	Auth-bewusste Hauptseite; öffentlicher Einstieg für Gäste und Weiterleitung für eingeloggte Nutzer.
src/app/api/substitutions/route.ts	API-Proxy mit Rate-Limit, Server-Cache und Fehlernormalisierung.
src/app/api/substitutions/route-utils.ts	Hilfsfunktionen für Datum, URL-Bildung und sichere Endpunktprüfung.
src/components/stundenplan/dashboard-client.tsx	Verbindet Datumsauswahl, Filter, Ergebnisanzeige und persönliche Nutzung im Dashboard.
src/hooks/use-substitutions.ts	Kapselt Datenabruf, lokalen Cache, Abort-Logik und Fehlerzustände im Browser.
src/lib/data-processing.ts	Normalisiert Rohdaten und enthält Sortier- und Filterlogik.
src/middleware.ts	Schützt kritische Pfade und setzt sicherheitsrelevante Header.
prisma/schema.prisma	Definiert das relationale Datenmodell für Nutzer, Stundenpläne, Subscriptions und Zustände.

7.3 Datenverarbeitung und Filterlogik

Eine zentrale Aufgabe der Anwendung besteht darin, die Rohdaten aus WebUntis in eine Form zu überführen, die für Nutzer sinnvoll lesbar ist. Dazu gehört die Einordnung des Vertretungstyps, das Sortieren nach sinnvollen Kriterien und das Filtern nach Suchbegriffen oder Kategorien. Gerade hier zeigte sich, dass die eigentliche Schwierigkeit oft nicht im Abruf selbst liegt, sondern in der fachlich sauberen Verarbeitung und Darstellung.

Zusätzlich spielt die Filterlogik eine Rolle für personalisierte Treffer. Erst wenn der eigene Stundenplan mit dem Tagesplan abgeglichen wird, kann die Anwendung unterscheiden, welche Vertretung nur allgemein vorhanden ist und welche für einen bestimmten Nutzer tatsächlich relevant ist.

7.4 Nutzerfluss und Bedienbarkeit

Ein wiederkehrendes Problem in den frühen Versionen war die fehlende Führung im Nutzerfluss. Technisch ließen sich einzelne Funktionen zwar ausführen, aber der Weg dorthin war nicht immer klar. Erst mit dem geführten Ablauf aus Registrierung, Admin-Setup, Onboarding und Dashboard entstand eine Nutzung, die auch für andere Personen unmittelbar verständlich war.

Diese Erfahrung war für mich besonders wichtig: Gute Bedienbarkeit entsteht nicht automatisch als Nebenprodukt funktionierenden Codes.[13] Sie ist selbst ein technisches Qualitätsmerkmal, weil sie Zustände, Rechte, Weiterleitungen und Fehlersituationen strukturiert ordnet.

7.5 Sicherheit, Datenschutz und Betrieb

Sicherheit war im Projekt kein eigener Zusatzblock am Ende, sondern ein durchgängiges Querschnittsthema. Geschützte Bereiche dürfen nur nach Authentifizierung erreichbar sein. Diese Zugriffskontrolle wird durch eine zentrale Middleware umgesetzt, die bei jedem Request prüft, ob der Pfad eine Authentifizierung erfordert, und unauthentifizierte Nutzer zur Anmeldeseite weiterleitet. Gleichzeitig setzt sie sicherheitsrelevante HTTP-Header wie Content Security Policy. Sensible Prüfungen laufen serverseitig, mutierende Requests werden abgesichert und die Auslieferung im Browser wird über Sicherheitsheader gehärtet. Zusätzlich war HTTPS für den produktiven Betrieb unverzichtbar, nicht nur wegen QStash, sondern auch wegen der generell sicheren Übertragung.

[13] Vgl. Krug (2014), Kap. 7: Gute Navigation und klare Einstiegspunkte als Grundvoraussetzung nutzbarer Webanwendungen.

Im Datenschutz blieb das Projekt bewusst auf einer technischen Ebene. Umgesetzt wurden etwa minimale Datenhaltung, Hashing sensibler Informationen, klare Rollentrennung und Pflichtseiten wie Impressum und Datenschutz. Ein formales juristisches DSGVO-Audit lag jedoch außerhalb des Projektumfangs.

7.6 Typische Schwierigkeiten und dessen Lösungen

1. Der Produktumfang war anfangs zu breit gedacht. Erst die frühen Versionen machten sichtbar, welche Funktionen wirklich zum Kernnutzen gehören. Die Lösung bestand in einer bewussten Rückführung auf den Nutzerkern: Datum, Vertretungen, Filter, persönlicher Bezug.
2. Der Nutzerfluss zwischen Registrierung, Rollenvergabe und persönlicher Nutzung war zunächst nicht konsistent. Mehrere Überarbeitungen waren nötig, bis der Ablauf technisch korrekt und gleichzeitig verständlich war.
3. Push-Benachrichtigungen erwiesen sich als deutlich komplexer als zunächst erwartet. Ohne Delta-Vergleich hätte das System dieselben Informationen immer wieder gesendet. Gelöst wurde das Problem durch Fingerprints, gespeicherte Zustände und einen periodischen Dispatcher.
4. Die responsiven Oberflächen verhielten sich auf unterschiedlichen Smartphones zunächst nicht stabil. Abgeschnittene Bereiche und verschobene Elemente wurden erst durch Rückmeldungen im Testbetrieb sichtbar und anschließend gezielt behoben.

7.7 Lessons Learned

Erstens habe ich gelernt, dass Produktqualität mehr ist als korrekte Logik. Eine gute Anwendung braucht klar definierte Zustände, sinnvolle Weiterleitungen und eine Oberfläche, die den Nutzer an den richtigen Stellen führt.

Zweitens wurde deutlich, dass vermeintlich attraktive Ziele wie Echtzeit nicht automatisch die bessere technische Lösung sind. Für dieses Projekt war Verlässlichkeit wichtiger als maximale Aktualisierungsfrequenz.

Drittens habe ich gelernt, dass Hintergrundprozesse in Webanwendungen fast immer zusätzlichen Zustand erfordern. Push-Benachrichtigungen werden erst dann wirklich nützlich, wenn sie nicht jeden Polling-Durchlauf ungefiltert nach außen weiterreichen.

Viertens zeigte sich, dass Hosting und öffentlicher Betrieb die Qualität einer Lösung sichtbar machen. Erst im produktiven Kontext werden Themen wie SSL, externe Scheduler, Browserberechtigungen und stabile Fehlerpfade wirklich relevant.

Schließlich war das Projekt auch eine Lektion in Gestaltungsdisziplin. Gute UI entsteht nicht durch frühes Design, sondern durch inhaltliche Klarheit. Erst nachdem Anforderungen und Nutzerfluss sauber standen, ließ sich die Oberfläche zielgerichtet verfeinern.[14]

8 Test und Ergebnisbewertung

8.1 Teststrategie

Die Qualitätssicherung stützt sich auf mehrere Ebenen. Automatisierte Tests prüfen die Kernlogik in Unit-, Integrations- und Komponententests.[15] Zusätzlich wurde die Anwendung im echten Gebrauch mit mehreren Personen und auf verschiedenen Geräten ausprobiert. Gerade diese Kombination war wichtig: Automatisierte Tests sichern die Logik, reale Nutzung deckt Probleme in Darstellung, Timing und Bedienbarkeit auf.

Zum Zeitpunkt dieser Dokumentation umfasst das Projekt 34 Testdateien mit insgesamt 134 Testfällen. Diese Tests gehören fest zur Entwicklungsroutine und werden nicht nur einmalig vor der Abgabe ausgeführt.

[14]Vgl. Nielsen (1994), Heuristik 2 „Match between system and the real world" und Heuristik 8 „Aesthetic and minimalist design": Gestaltung folgt dem Verständnis der Nutzerbedürfnisse, nicht umgekehrt.
[15]Vgl. Spillner/Linz (2019), Kap. 2–3: Teststufen (Unit, Integration, Komponente) und ihre Abgrenzung im Testprozess.

8.2 Ausgewählte Testfälle

Testziel	Typ	Vorgehen	Ergebnis
Vertretungstyp-Erkennung	Unit	CSS-Klassen und Textmuster an die Verarbeitungsfunktion übergeben.	Korrekte Klassifizierung für Entfall, Vertretung und Verlegung.
Rate-Limiting der API	Integration	61 Anfragen an /api/substitutions senden.	Ab Anfrage 61 erfolgt 429 mit Retry-After-Header.
Retry bei Upstream-Fehler	Integration	503-Antwort der Datenquelle simulieren.	Mehrere Wiederholungsversuche mit Backoff werden korrekt ausgelöst.
Delta-Logik für Push	Unit	Identische und geänderte Fingerprints vergleichen.	Aktion "skip" bei unverändertem, "send" bei geändertem Stand.
Bootstrap-Admin	Integration	Ersten Nutzer mit und ohne konfigurierte ADMIN_EMAILS registrieren.	Nur erlaubte E-Mail erhält Admin-Rolle; Schutzmechanismen greifen.
Stundenplan-Überlappung	Integration	Überlappende Einträge in die Stundenplan-Route senden.	Ablehnung im strikten Modus, kontrollierte Akzeptanz mit Flag.
Dashboard-Datumsdialog	Komponente	Datumsauswahl in der Oberfläche auslösen.	Anzeige aktualisiert sich auf das neue Datum.

8.3 Soll-Ist-Abgleich der SMART-Ziele

ID	Status	Soll	Ist-Nachweis
S1	Erfüllt	Vertretungsinfos in maximal drei Interaktionen.	Datumsauswahl und Ergebnisanzeige gelingen ohne zusätzlichen Seitenwechsel; praktisch meist in zwei Klicks.
S2	Erfüllt	Kombinierte Suche und Kategorienfilter stabil nutzbar.	Filter- und Suchlogik sind in der Datenverarbeitung umgesetzt und im Frontend eingebunden.
S3	Erfüllt	API robust bei Last und Upstream-Fehlern.	Rate-Limit, Retry-Logik, Server-Cache und definierte Fehlerantworten sind implementiert und getestet.
S4	Erfüllt	Sicherheits- und Datenschutzbasis aktiv.	Geschützte Bereiche, Sicherheitsheader und Pflichtseiten sind vorhanden.
S5	Weitgehend erfüllt	Wartbarkeit und Nachvollziehbarkeit absichern.	Tests für Kernpfade und dokumentierte Struktur vorhanden; zusätzliche UI-Randfälle bleiben als Ausbaupotenzial.

Insgesamt sind die Kernziele S1 bis S4 erreicht. Das Ziel S5 ist in weiten Teilen umgesetzt, bleibt aber bewusst offen für weitere Verfeinerung - etwa durch zusätzliche Tests für seltene Randfälle, weitere Dokumentation einzelner UI-Zustände und einen noch breiteren Praxistest.

8.4 Grenzen der Lösung

Trotz des positiven Ergebnisses hat die Anwendung klare Grenzen. Sie ist auf die Friedrich-Dessauer-Schule und deren konkrete Datenquelle zugeschnitten. Ein Mehrschulbetrieb wäre nur mit separater Konfiguration und zusätzlicher Administrationslogik sinnvoll. Außerdem handelt es sich um eine Webanwendung und nicht um eine native App. Das ist für den Einsatzzweck passend, setzt aber Browserunterstützung und Web-Push-Fähigkeit voraus.

Auch im Datenschutz bleibt die Lösung technisch und nicht juristisch bewertet. Die wichtigsten technischen Schutzmaßnahmen sind vorhanden, aber eine formale rechtliche Prüfung wurde im Rahmen dieser BLL nicht durchgeführt.

9 Fazit und Ausblick

9.1 Zusammenfassung

Das Projekt hat sein zentrales Ziel erreicht: Aus einer schwer nutzbaren Ausgangsanzeige ist eine eigene Webanwendung entstanden, die Vertretungsdaten schneller, verständlicher und in personalisierter Form bereitstellt. Die Arbeit zeigt dabei nicht nur das Endergebnis, sondern vor allem die technische Entwicklung vom frühen Prototyp bis zum stabilen Webprodukt.

Besonders wichtig ist aus meiner Sicht, dass die Eigenleistung nicht in einzelnen Framework-Entscheidungen liegt, sondern in der Kombination mehrerer selbst entwickelter Bausteine: der Analyse der Datenquelle, dem API-Proxy mit Cache und Fehlerpfaden, dem Stundenplanabgleich und der Delta-basierten Push-Logik. Erst dieses Zusammenspiel macht den Unterschied zwischen einer bloßen Oberfläche und einer echten Anwendung.

9.2 Weiterentwicklung

Der nächste sinnvolle Schritt wäre ein längerer Praxistest mit einer größeren und heterogeneren Nutzergruppe. Dadurch ließen sich Rückmeldungen nicht nur vereinzelt, sondern systematisch erfassen. Auf dieser Basis wären feinere Personalisierungen, eine bessere Priorisierung wichtiger Änderungen und zusätzliche Auswertungen zum Erfolg oder Misserfolg von Push-Benachrichtigungen denkbar.

Auch im Administrationsbereich bestehen Ausbauoptionen. Dazu gehören eine noch einfachere Pflege von Lehrerkürzeln, verbesserte Nutzerverwaltung und eine klarere Trennung zwischen produktiven Einstellungen und Testfunktionen. Denkbar wäre langfristig außerdem eine noch stärkere Entkopplung zwischen externer Datenquelle und eigener Verarbeitung, sofern sich verlässlichere oder dokumentierte Zugriffswege ergeben.

9.3 Persönlicher Lerngewinn

Der größte Lerngewinn lag für mich nicht in einem einzelnen Feature, sondern im Zusammenspiel von Produktdenken und technischer Umsetzung. Ich habe gelernt, dass gute Software nicht nur aus funktionierendem Code besteht, sondern aus klaren Anforderungen, sauberer Struktur, geeigneten Sicherheitsmaßnahmen, nachvollziehbaren Tests und ehrlicher Reflexion über Grenzen.

Gerade für eine Besondere Lernleistung ist das ein wichtiger Punkt: Informatik zeigt sich hier nicht in theoretischen Definitionen allein, sondern in begründeten Entscheidungen unter realen

Randbedingungen. Genau diesen Prozess wollte ich mit der vorliegenden Ausarbeitung sichtbar machen.

10 Quellenverzeichnis

10.1 Projektinterne Quellen

- Repository des Projekts fds-vertretungsplan (Code, Tests, Konfiguration, Dokumentationsentwürfe).
- Eigene Browser-Netzwerkanalyse der WebUntis-Vertretungsanzeige der Friedrich-Dessauer-Schule.
- Projektgespräche und Rückmeldungen mit den betreuenden Lehrkräften sowie Testfeedback aus dem Informatik-Kurs.

10.2 Technische Dokumentationen

- Next.js - https://nextjs.org/docs
- React - https://react.dev
- TypeScript - https://www.typescriptlang.org/docs/
- Prisma - https://www.prisma.io/docs
- Vitest - https://vitest.dev/
- PostHog - https://posthog.com/docs
- Upstash QStash - https://upstash.com/docs/qstash

10.3 Webstandards und Sicherheitsquellen

- RFC 8292 - Voluntary Application Server Identification (VAPID) for Web Push
- RFC 5861 - HTTP Cache-Control Extensions for Stale Content
- W3C Push API
- W3C Service Workers
- W3C Content Security Policy Level 3
- OWASP Top 10 (2021)

10.4 Fachliteratur

- Sommerville, Ian: Software Engineering. 10. Auflage, Pearson, 2015.

- Krug, Steve: Don't Make Me Think - A Common Sense Approach to Web Usability. 3. Auflage, New Riders, 2014. (Usability-Prinzipien, nutzerorientierte Gestaltung, iteratives Interface-Design)
- Spillner, Andreas / Linz, Tilo: Basiswissen Softwaretest. 6. Auflage, dpunkt.verlag, 2019. (Testmethodik, Teststufen, Testarten und Testprozess)
- Nielsen, Jakob: 10 Usability Heuristics for User Interface Design. Nielsen Norman Group, 1994 (aktualisiert 2024), https://www.nngroup.com/articles/ten-usability-heuristics/ (abgerufen am 11.02.2026).

10.5 Organisatorische Quellen

- Besondere Lernleistung im Landesabitur 2026 in Hessen.
- Lehrer-Notizen zu Aufbau, Inhalt und formalen Vorgaben der schriftlichen Ausarbeitung